Comprehensive Order Dependency List in Java

Elements that require declaration order (must be declared before use):

1. **Field Initializers**
   - Depend on: *Fields* declared above

```
int a = b;  // Requires b declared above
int b = 10;
```

2. **Static Field Initializers**
   - Depend on: *Static fields* declared above

```
static int x = y;  // Requires y declared above
static int y = 5;
```

3. **Instance Initialization Blocks**
   - Depend on: *Fields* declared above

```
{ System.out.println(z); }  // Requires z declared above
int z = 20;
```

4. **Static Initialization Blocks**
   - Depend on: *Static fields* declared above

```
static { System.out.println(count); }  // Requires count declared above
static int count = 0;
```

5. **Enum Constant Arguments**
   - Depend on: *Enum constants* or *static fields* declared above

```
enum Size {
    SMALL(1),
    MEDIUM(SMALL.code+1);  // Requires SMALL declared above
    final int code;
    Size(int code) { this.code = code; }
}
```

6. **Annotation Default Values**
   - Depend on: *Constant fields* declared above

```
static final String DEFAULT = "test";
@interface MyAnnotation {
    String value() default DEFAULT;  // Requires DEFAULT declared above
}
```

7. **Annotation Arguments**
   - Depend on: *Constant fields* declared above

```
static final int MAX = 100;
@Limit(MAX)  // Requires MAX declared above
int value;
```

8. **Annotation Type Declarations**
   - Depend on: *Annotation types* declared before use

```
@interface Special {}  // Must be declared before use
@Special
void method() {}
```

9. **Enum Types in Annotations**

- Depend on: *Enum types* declared before use

```
enum Status { ACTIVE }  // Must be declared first
@State(status = Status.ACTIVE)
void execute() {}
```

10. **Generic Type Bounds**
    - Depend on: *Types* declared before use

```
class A {}
class B<T extends A> {}  // Requires A declared above
```

Order-Independent Elements

Can reference any member regardless of position:

1. **Method Bodies**

```
void methodA() { methodB(); }  // Valid even if methodB declared later
void methodB() {}
```

2. **Constructor Bodies**

```
MyClass() { this.value = 42; }  // Valid even if value declared later
int value;
```

3. **Inner Classes**

```
class Inner { void use() { outerField = 10; } }  // Valid
int outerField;
```

4. **Interface Default Methods**

```
interface MyInterface {
    default void log() { System.out.println(MSG); } // Valid
    String MSG = "Hello";
}
```

Complete Dependency Matrix

Dependent Element	Dependency Type	Order Requirement	Example
Field initializer	Same-class field	✅ Above	`int a = b`
Static field initializer	Same-class static field	✅ Above	`static int x = y`
Instance init block	Same-class field	✅ Above	`{ System.out.println(field); }`
Static init block	Same-class static field	✅ Above	`static { x = STATIC_FIELD; }`
Annotation default	Same-class constant	✅ Above	`default VALUE`
Annotation argument	Same-class constant	✅ Above	`@Anno(CONST)`
Annotation type usage	Annotation declaration	✅ Above	`@MyAnnotation`
Enum in annotation	Enum declaration	✅ Above	`status = Status.ACTIVE`
Generic type bound	Class/interface	✅ Above	`T extends MyType`
Method body	Any member	❌ None	`void x() { y(); }`
Constructor body	Any member	❌ None	`Foo() { this.field = value; }`
Inner class	Any enclosing member	❌ None	`class Inner { void use() { outer(); } }`
External references	Cross-class elements	❌ None	`OtherClass.CONST`

This comprehensive list covers all order-sensitive and order-insensitive dependencies in Java class declarations. Use it as a reference for code analysis, refactoring, and AST manipulation tasks.